

Paso 8 - Arreglos | Ejercicios

Acuérdate: los casilleros empiezan en `0`, el último es `length - 1`, y `length` va **sin** paréntesis.

Para imprimir un arreglo completo necesitas `import java.util.Arrays;` y `Arrays.toString(arreglo)`.

► **Plantillas:** [plantillas/](#) — un `.java` por ejercicio, con el enunciado, la salida esperada y los `// TODO` donde va tu código. Si las usas, el archivo ya viene creado (`B8_1_CincoNotas.java`) y **ese** nombre manda sobre el que diga el enunciado.

✓ **Solucionario:** [soluciones/](#) — el código de cada ejercicio, la salida real y por qué se resuelve así. Ábrelo **después** de intentarlo.

Cómo se lee cada ejercicio: el título dice qué hay que lograr; el bloque de abajo es la **salida esperada**, o sea lo que tiene que aparecer en tu pantalla (en los programas que preguntan cosas, lo que va después de la pregunta es lo que **escribe el usuario**); las citas como esta son pistas. Si no sabes por dónde partir, abre la plantilla: los `// TODO` numerados son los pasos, en orden. Compila y ejecuta después de cada uno.

B8.1 - Las cinco notas

Qué practicas: llenar un arreglo con datos del teclado y recorrerlo para calcular promedio, máximo, mínimo y un conteo, todo en un solo `for`.

Pide 5 notas por teclado, guárdalas en un `double[]` y muestra:

```
Notas: [5.5, 6.2, 3.8, 7.0, 4.1]
Promedio: 5.3
Nota mas alta: 7.0
Nota mas baja: 3.8
Notas rojas: 1
```

Junta el paso 4 (leer), el paso 7 (el `for` que llena el arreglo) y el paso 8. Para la más baja, parte suponiendo que es la primera y ve comparando.

B8.2 - Al revés

Qué practicas: recorrer un arreglo en reversa, y modificarlo de verdad intercambiando posiciones simétricas.

Con el arreglo `{10, 20, 30, 40, 50}`:

1. Imprímelo al revés, recorriéndolo con un `for` que va de atrás para adelante.
2. Ahora **dalo vuelta de verdad**: que el arreglo quede `{50, 40, 30, 20, 10}`, intercambiando el primero con el último, el segundo con el penúltimo, etc.

```
Original:  [10, 20, 30, 40, 50]
Invertido: [50, 40, 30, 20, 10]
```

Para el segundo necesitas el intercambio del ejercicio B3.5, y recorrer **solo la mitad** del arreglo. ¿Qué pasa si lo recorres entero? Pruébalo.

B8.3 - Buscar

Qué practicas: una búsqueda lineal en un arreglo, usando `-1` como valor centinela para "no encontrado".

Parte con este arreglo ya escrito en el código:

```
int[] numeros = {10, 20, 30, 40, 50, 60};
```

Pide un número por teclado y responde en qué posición está, o que no está.

```
Buscar: 30
Encontrado en la posicion 2 (el tercero)
```

```
Buscar: 99
No esta en la lista
```

Pista: una variable `int posicion = -1;` antes del `for`, y `break` apenas lo encuentres. El `-1` es la forma clásica de decir "no está".

B8.4 - El histograma

Qué practicas: recorrer un arreglo y, por cada valor, dibujar una cantidad proporcional de símbolos con un `for` anidado.

Con 5 notas en un arreglo, dibuja una barra de asteriscos por cada una:

```
5.5 | *****
6.2 | *****
3.8 | ***
7.0 | *****
4.1 | ****
```

La cantidad de asteriscos es la parte entera de la nota: un cast a `int`. Un `for` recorre el arreglo y otro dibuja los asteriscos.

B8.5 - La lista de ramos (`ArrayList`)

Qué prácticas: usar un `ArrayList` (`add`, `get`, `size`, `remove`, `contains`) dentro de un menú, y compararlo con un arreglo de tamaño fijo.

Escribe un programa con un menú (paso 6) que administre tus ramos en un `ArrayList<String>`:

```
1) Agregar ramo
2) Listar ramos
3) Cuantos ramos
0) Salir
```

Usa `ramos.add(...)`, `ramos.size()` y `ramos.get(i)`.

Después busca en internet qué hacen `ramos.remove(0)` y `ramos.contains("P00")`, y agrega las opciones para borrar y para buscar.

Compara con el arreglo: ¿podrías haber hecho este menú con un `String[]`? ¿Qué tendrías que haber decidido antes de empezar?

B8.6 - Desafío: sin repetidos

Qué prácticas: recorrer la parte ya llena de un arreglo antes de agregar un valor nuevo, para rechazar repetidos.

Pide 5 números y guárdalos en un arreglo, pero **rechaza los repetidos**: si el usuario escribe uno que ya está, se lo dice y se lo vuelve a pedir.

```
Numero 1: 7
Numero 2: 3
Numero 3: 7
    El 7 ya esta en la lista
Numero 3: 9
```

Necesitas un `for` que recorra lo que ya llevas guardado cada vez que entra un número nuevo, dentro de un `while` que insiste.

Cuando termines, sigue con el [paso 9](#).